

Ishaan Goyal

ishaan6@illinois.edu | [Website](#) | [LinkedIn](#) | [GitHub](#) | [Twitter](#)

EDUCATION

University of Illinois Urbana-Champaign

Bachelor of Science in Computer Science

Relevant Coursework: Data Structures & Algorithms, Operating Systems, Systems Programming, Distributed Systems, Advanced Computer Architecture, Linear Algebra, Parallel Programming, ML Systems, Generative AI

Urbana - Champaign, IL

May 2027

EXPERIENCE

Founding Engineer, ML

A Vinyl Bar in Shibuya | Python, PyTorch, MLX, Quantization, On-Device Inference

Feb 2026 – Aug 2026

Remote

- Collaborated directly with **Spotify's former Head of Innovation** at a **\$50M AI x Music startup** to build production-oriented audio ML systems across **music search, stem separation, and on-device inference**.
- Built an **end-to-end music ETL pipeline** ingesting **1M+ songs** into the internal database, normalizing metadata and audio features for **search/retrieval** while preparing curated datasets for downstream **fine-tuning and evaluation**.
- Optimized BS-RoFormer** for on-device deployment by retrofitting a **sparse attention scheme**, achieving **9.65 avg SDR on MUSDB18** while significantly reducing inference latency through **weight quantization** and **model fine-tuning**.
- Improved **operational efficiency** by building an **agentic workflow harness** that automated multi-step processes across data ingestion, evaluation, and content operations, reducing manual intervention by **60%** and accelerating task completion by **2.5×**.

Compiler Researcher

ADAPT Lab, UIUC | JIT, Caching, GPU Architecture, Compiler Optimization

May 2025 – Aug 2025

Champaign, IL

- Co-authored research paper** on ConstraintFlow DSL compiler optimizations for neural network verification; refactored core IR passes & created 15+ technical diagrams improving codebase comprehension.
- Engineered multi-level caching system and JIT compilation for tensor-intensive kernels, achieving **51× speedup** in critical hot-path operations through memory hierarchy exploitation.
- Designed **CPU-GPU synchronization blueprint** and GPU-scaling roadmap for multi-GPU clusters.
- Built granular profiling instrumentation across 20+ compilation passes, identified 3 dominant bottlenecks, and reduced end-to-end compile time by **37%**.

PROJECTS

JAX Transformer + MoE Kernel Optimization | JAX, Flax, Optax, TPU, Pallas, MoE

Jun 2026

- Built and trained a **≈10M parameter decoder-only Transformer from scratch** in **JAX/Flax/Optax** to study algorithmic reasoning on **up-to-3-digit addition**, implementing the full pipeline across **synthetic dataset generation, tokenization, batching, causal masking, train/eval loops, checkpointing, exact-match evaluation**, and generalization analysis by sequence length, carry behavior, and out-of-distribution arithmetic difficulty.
- Derived **Chinchilla-style scaling estimates** for dense vs. **MoE Transformers**, comparing active parameters, total parameters, token budget, routing overhead, and compute-optimal tradeoffs for sparse expert architectures.
- Extended the dense feedforward block into a **top-1 MoE layer** with router logits, token dispatch/combine logic, expert-capacity handling, and `jax.lax.ragged_dot` expert execution to compare dense vs. sparse Transformer training behavior.
- Wrote a custom **Pallas fused up/down projection kernel** for **F > D**, reducing intermediate expert-activation memory traffic and achieving **≈1.3× MoE forward-pass speedup** over the ragged-dot baseline.

GPT-2 Inference Optimization on HPC Cluster | CUDA, C++, Slurm, Nsight

Oct 2025 – Dec 2025

- Profiled GPT-2 inference on NVIDIA A40; identified **matmul kernel as 97–99% of GPU time** with severe underutilization (**1–4% peak compute**) due to small-grid launch and low occupancy.
- Reduced matmul latency by **10–13% (2–3% E2E speedup)** by increasing CTA count and applying **SMEM tiling, loop unrolling, & vectorized global loads** to improve data reuse and memory coalescing.
- Increased effective **Tensor Core utilization** by implementing **WMMA GEMM (FP16 inp. FP32 acc.)** with **Split-K parallelization & double buffering**, overlapping global→shared transfers with compute to reduce scoreboard stalls and raise SM issue rate.
- Reduced decode-time compute and memory traffic by introducing **KV caching**, lowering autoregressive complexity from **O(T²)** to **O(T)** and preventing per-token latency from scaling with sequence length; additionally explored **local/windowed attention** to reduce attention memory bandwidth requirements.

VidSmith (GitHub) | Go, Kubernetes, AWS EKS, RabbitMQ, FFmpeg

Jul 2025

- Deployed containerized gateway, transcoder, captioning, moderation, and messaging services to **Kubernetes on AWS EKS**, defining rolling updates, readiness/liveness probes, resource limits, pod anti-affinity, secrets/configuration, and TLS ingress.
- Designed an **event-driven media pipeline** across a TypeScript gateway and Go services, using RabbitMQ for asynchronous coordination and Amazon S3 for durable source and rendition storage.
- Built an **FFmpeg transcoder** that produced 1080p/720p/480p HLS renditions and thumbnails, parallelized multipart S3 transfers, and delivered media through **CloudFront** with end-to-end processing status.

TECHNICAL SKILLS

Languages: C, C++, CUDA C++, Python, Go, Java, JavaScript/TypeScript, Verilog, Bash, SQL

ML/AI Systems: PyTorch, JAX, Flax, Optax, TensorFlow, Hugging Face, MLX, Core ML, LoRA/PEFT, MoE, MTP, Speculative Decoding, KV Cache, FlashAttention, Quantization, RLHF, Pallas

Inference/Kernel Optimization: CUDA, cuBLAS, cuDNN, NCCL, WMMA, Tensor Cores, Pallas kernels, KV-cache management, batching, token routing, prompt-lookup decoding, INT4/INT8/FP8 quantization

Systems/Compiler/Infra: Linux, Slurm, Docker, Kubernetes, Git, CMake, gdb, perf, Nsight, nsys, JIT compilation, caching, profiling, OpenMP, MPI, AWS, GCP, Redis, PostgreSQL, Elasticsearch, Kafka, gRPC, Protobuf